

# Data Preprocessing in Linear Regression (Complete Beginner Guide)

## Why Preprocess for Linear Regression?

Linear Regression assumes:

- The relationship between features and target is **linear**
- The data has **no missing values**
- Features are **numerical and scaled**
- There are **no extreme outliers**
- Features are **independent of each other**

So we **must preprocess the data** properly before applying the model.

---

## Step-by-Step Guide with Explanation + Code

Let's walk through each step.

---

### 1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

---

### 2. Load and Explore the Dataset

```
df = pd.read_csv("data.csv")
print(df.head())
print(df.info())
```

This helps you:

- See data types (numerical, categorical)
  - Check for missing values
  - Understand the structure of your data
-

### 3. Handle Missing Values

#### a. Check for missing values:

```
print(df.isnull().sum())
```

#### b. Fill or drop:

```
df.fillna(df.mean(), inplace=True) # For numerical columns  
# or  
df.dropna(inplace=True) # Remove rows with missing values
```

---

### 4. Encode Categorical Variables

Linear regression requires **numeric values**.

```
le = LabelEncoder()  
df['gender'] = le.fit_transform(df['gender']) # Male:1, Female:0
```

Or use One-Hot Encoding for more than 2 categories:

```
df = pd.get_dummies(df, drop_first=True)
```

---

### 5. Feature Selection

Choose relevant features (independent variables  $x$ ) and target variable ( $y$ ).

```
X = df[['age', 'salary', 'gender']] # Example features  
y = df['house_price'] # Target variable
```

---

### 6. Train-Test Split

Split the data to train and test the model.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- 80% for training, 20% for testing
  - `random_state=42` gives reproducible results
- 

### 7. Feature Scaling (Standardization)

Linear regression is sensitive to different scales of features.

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

This step standardizes all features to have **mean = 0** and **standard deviation = 1**.

---

## 8. Train the Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

The model finds the best line to fit the data by minimizing the **error**.

---

## 9. Make Predictions

```
y_pred = model.predict(X_test)
```

---

## 10. Evaluate the Model

Use metrics to check how good the model is.

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

- **MSE (Mean Squared Error):** Measures average squared error between predicted and actual values.
  - **R<sup>2</sup> (R-squared):** Measures how well the model explains the variance in the data. Closer to 1 = better.
- 



## Complete Code Together:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("data.csv")

# Clean missing values
df.fillna(df.mean(), inplace=True)

# Encode categorical variable
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])

# Select features and label
X = df[['age', 'salary', 'gender']]
y = df['house_price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))
```

---

## Summary of Preprocessing Steps in Linear Regression:

| Step                      | Purpose                                     |
|---------------------------|---------------------------------------------|
| Handle missing values     | Remove or fill null values                  |
| Encode categorical values | Convert strings to numbers                  |
| Feature selection         | Choose relevant features for prediction     |
| Feature scaling           | Normalize data for better model performance |
| Train-test split          | Avoid overfitting and test generalization   |

---

Would you like:

- This explanation with sample **CSV data**?
- To apply this in **Jupyter Notebook** with charts (like correlation heatmap, residual plots)?
- Or learn **polynomial regression or multiple linear regression** next?

Let me know!